

Draco 2: An Extensible Platform to Model Visualization Design

Junran Yang*
University of Washington,
Seattle

Péter Ferenc Gyarmati†
University of Vienna,
Vienna

Zehua Zeng‡
University of Maryland,
College Park

Dominik Moritz§
Carnegie Mellon University,
Pittsburgh

ABSTRACT

Draco introduced a constraint-based framework to model visualization design in an extensible and testable form. It provides a way to abstract design guidelines from theoretical and empirical studies and applies the knowledge in automated design tools. However, Draco is challenging to use because there is limited tooling and documentation. In response, we present Draco 2, the successor with (1) a more flexible visualization specification format, (2) a comprehensive test suite and documentation, and (3) flexible and convenient APIs. We designed Draco 2 to be more extensible and easier to integrate into visualization systems. We demonstrate these advantages and believe that they make Draco 2 a platform for future research.

Index Terms: Human-centered computing—Visualization—Visualization systems and tools

1 INTRODUCTION

For the remainder of this paper, we refer to the original system as Draco 1, while Draco and Draco 2 denote our presented contribution.

Draco 1 [16] aims to make design guidelines concrete, actionable, and testable. By encoding guidelines as logical rules, developers and researchers can build computational knowledge bases for automatically assessing existing charts [1, 7, 15] and generating new recommended charts [13, 21–23]. To evaluate and recommend charts, Draco 1 uses a constraint solver that checks whether a specification is valid or finds a design that incurs a minimal cost of constraint violations. Using this approach, Draco 1 can warn users about ineffective designs and provide design suggestions, even for partial and ambiguous requests. By formalizing design guidelines as constraints, visualization researchers and practitioners can add their own design considerations and immediately test the results, seeing how different rules and weights change which visualizations are considered the most preferable. The weights of the constraints in the Draco 1 knowledge base can be adjusted by hand or learned from graphical perception results [24, 25, 27] using machine-learning [8, 11].

Since its introduction in 2018, Draco 1 has become a popular tool in the visualization research community. Yet, even though Draco 1 as a conceptual framework can keep up with expanding design rules over time, its original implementation and knowledge base are limited in two major ways. First, Draco 1 only focuses on single views and faceted views (using row and column encodings) limiting its support for multi-view visualizations. Its specification language is also closely tied to Vega-Lite [17]. It is not generalizable to render recommended results with other visualization libraries. Second, Draco 1 comes with limited documentation, low test coverage, and limited tooling support, making it difficult to adopt it as a library. The inability to be integrated into emerging projects hinders Draco 1 from being a true platform for future research in

visualization recommendation and reasoning about visualization design.

To overcome these limitations, we introduce Draco, an improved system for capturing and applying visualization design best practices. In summary:

- Draco introduces an *improved visualization specification format*, independent of Vega-Lite, which makes it more flexible and extensible. It supports much more designs, including multi-layer and multi-view visualizations and makes scales a first-class concept.
- Draco (1) has *thorough documentation* covering the core modules, lower-level building blocks, and examples, and (2) a comprehensive test suite with *100% unit test coverage*, making it more reliable and suitable for adoption. Draco is easier to set up as it runs entirely in Python (the original Draco 1 system needed Python and JavaScript). We provide a *REST API* so Draco can still be integrated into web-based applications. We also distribute Draco as a *WebAssembly* package which runs in any modern browser.
- Draco has convenient APIs to interact with the knowledge base to convert constraints from and to a nested format, validate specifications, recommend optimal completions of partial specifications (visualization recommendation), and to debug, adapt, and extend the knowledge base. We consider the default knowledge base in Draco as a starting point that researchers and systems builders adapt using these tools.

We believe that our work is a significant update to the original Draco 1 system. We envision that with the improvements described in this paper, Draco 2 can be a solid platform for customization and future visualization research. Towards this goal, we make Draco 2 available as open source at github.com/cmdug/draco2. In this paper, we discuss the version 2.0.0.

2 BACKGROUND AND RELATED WORK

Draco upgrades its previous version, Draco 1 [16]—a framework for efficiently modeling visualization design knowledge towards constructing new visualization recommendation algorithms, with a more general visualization specification and more user-friendly utilities for knowledge representation.

Visualization Specification: Automated visualization tools use specification languages to describe and synthesize visualization designs. Mackinlay’s APT [13] system ranks encoding choices based on the *expressiveness* and *effectiveness* criteria. Tableau’s ShowMe [14] suggests specific encodings with heuristic rules. Voyager’s [22, 23] CompassQL [21] and Draco 1 both build on the Vega-Lite [17] grammar and combine rules that model fine-grained design knowledge with hand-tuned scores. In Draco, we re-design the logical representation to a generalized and extended chart specification format that is extensible and renderer-agnostic. With such a format, we support multiple views and view composition.

Modeling Visualization Design Knowledge: Visualization recommendation researches on algorithms including rule-based methods considering theoretical principles [13, 14, 22, 23] or proposing new metrics [2, 9, 19], and ML-based approaches [8, 11, 12] learning from a vast corpus of empirical results. Visualization recommendation frameworks [16, 18, 21] have been proposed to make it easier to design and test new recommendation algorithms. Draco 1 is the only one that was designed with the explicit goal of being extensible

*e-mail: junran@cs.washington.edu

†e-mail: peter.ferenc.gyarmati@univie.ac.at

‡e-mail: zhzeng@umd.edu

§e-mail: domoritz@cmu.edu

and adaptable. It allows modeling visualization design knowledge in Answer Set Programming (ASP) and uses the Clingo solver [4–6] to search the constrained space and rank the answers with weighted costs. Draco aims to provide a platform for future research beyond constructing visualization recommendation algorithms. For example, Zeng et al. [26] use Draco to analyze the implication of different graphical perception studies.

3 COMPONENTS OF DRACO

Draco has three main components: a general description language for charts, a knowledge base that encodes best practices using hard and soft constraints, and an API to manipulate the knowledge base and programmatically reason about the knowledge base using a constraint solver.

3.1 Draco Specification Format

To express knowledge over visualization designs, Draco describes visualizations as logical *facts* similar to Draco 1. Draco specifies charts in a more generic and extensible way than its previous version. In general, it describes the structure of charts as nested specifications. Since Clingo needs a flat list of facts, Draco has methods to convert the nested representation to (`dict_to_facts`) and from (`answer_set_to_dict`) a flat list of logical facts with the relationships also expressed as logical facts.

3.1.1 Nested specification format with Entities and Attributes



Figure 1: (a) Chart specification as a nested dictionary (left, here as a Python `dict`), (b) flattened list of logical facts (middle, in Answer Set Programming), (c) the rendered visualization result, and (d) the skeleton abstracted from the logical format.

Draco specifies the nested chart specification with two kinds of facts: *entity* and *attribute* as shown in Figure 1. Entities describe objects and their association with unique identifiers, while attributes describe the properties of entities. For example, `entity(mark,v0,m1). attribute((mark,type),m1,bar).` specifies a mark object `m1` of type `bar` on the view object `v0`.

The logical format can be seen as a tree where the entities and attributes are the nodes connected by the entity keys. For example, Figure 1 (d) shows the skeleton of the specification with only the entities. Using this format, Draco allows for both complete and partial specification as input. A complete specification has attributes associated with each entity, specifying a ready-to-render chart. A partial specification defines either part of all components in a complete chart (e.g., one layer of a multi-layer chart), the skeleton of a chart without the attributes (e.g., a single-view-single-layer chart with an arbitrary mark type), or a mix of both. Our enumeration algorithm supported by ASP augments the skeleton with additional entities and completes it by filling in potential attributes. Meanwhile, Draco reasons about fine-grained rules related to subtrees of the input by checking if they

satisfy the given constraints. Additional hints can be included as part of the query to constrain the target outputs. For example, adding aggregate rules like `:- {entity(encoding,_,_)} <= 2.` filters out designs with less than three encodings (headless rules are integrity constraints that derive false from their body, and satisfying the body results in a contradiction, which is disallowed). The tree structure of logical format allows for querying, searching, and reasoning about abstract visualization composition.

The dictionary format is an abstraction from the logical format that can generalize to multiple visualization specification languages with customized renderers. Its compact format keeps the structural information but is agnostic of entity identifiers. Thus, it deduplicates structural equivalent specifications whose entity keys are different. For instance, `entity(view,root,v0). attribute((view,coords,v0,polar))` and `entity(view,root,0). attribute((view,coords,0,polar))` would both be represented as `{"view": [{"coords": "polar"}]}` in a dictionary format since they are structurally identical, even though the entity keys `v0` and `0` are different.

3.1.2 Encoding Visualizations in Draco

Draco 1 uses an encoding based on the Grammar of Graphics (GoG) [20] and Vega-Lite [17]. However, Draco 2 is not limited to the features Vega-Lite supports. For instance, in Vega-Lite and Draco 1, each encoding has a data type that describes the semantics of the data (quantitative, temporal, ordinal, or nominal). However, encoding data types are omitted in Draco 2 because they can be automatically inferred from the combination of primitive field type (number, string, etc.) and the scale type (linear, log, ordinal, or categorical) for the encoding. Therefore, Draco 2's constraints directly reason about primitive field type and scale type, which are both explicit elements in visualizations compared to encoding type. Draco 2 also makes scales an explicit entity that can be associated and shared with multiple encodings across marks. Shared scales allow for comparisons across marks. Vega-Lite, which does not explicitly make scales independent entities, has to resort to a mechanism in which authors specify how scales in a view resolve¹.

Because of the nested format with entities and attributes, Draco 2 is generic and convenient to extend. Figure 1 shows the complete specification for a single-view single-layer bar chart. However, a Draco 2 program can encode visualizations in multiple views, where a view can contain one or more marks that encode data and corresponding scales. If a view has multiple marks, Draco 2 assumes that the marks are in the same view space in the chart (i.e., layered). Besides the visualization, a Draco 2 program can describe the data schema and the primary visualization task. This format could easily be extended with additional attributes and entities. For instance, one could add additional attributes about a mark such as the font size, scale, or color scheme. New entities could be legends and axes so that Draco 2's constraints could then reason about the position, size, or other properties of these guides.

3.2 Knowledge Base

Just like its predecessor, Draco uses a collection of hard and soft constraints over the logical facts to represent design knowledge guidelines. While the hard constraints span the space of all designs considered valid, the soft constraints define the preferences over the space to rank these designs. When a user queries with a partially specified visualization, Draco eliminates ill-formed (e.g., using the encoding channel *shape* for a mark that is not *point*) or non-expressive (e.g., aggregating *ordinal* fields with *mean* function) designs with the hard constraints and searches the design space for the lowest-cost specifications. The *Draco cost* of a visualization is the weighted sum of the costs of all violated soft constraints. And the weights reflect the relative importance of each violation in the

¹vega.github.io/vega-lite/docs/resolve.html

total cost. As a starting point, the default knowledge base consists of constraints adopted from CompassQL rules.

There are two ways to obtain soft constraint weights. First, our API allows algorithm designers to define their own sets of soft constraints and manually assign a weight to each constraint to indicate their preferences. Second, Draco-Learn can learn weights for existing soft constraints from ranked pairs of visualizations (the learning algorithm is the same as in Draco 1 [16]). These pairs could come from different experimental studies or theoretical rankings.

Draco loads and exposes the knowledge base as answer-set programs. We use the **definitions** programs to declare the domains of visualization attributes. For example, `domain((mark,type), (point;bar;line;area;text;tick;rect))`, defines the choices of mark types. To enforce the search space to follow the correct Draco general description language, we use the **constraints** programs. `violation(invalid_domain) :- attribute(P,_,V), domain(P,_,), not domain(P,V).`, for example, allows only valid domain values following the definitions. Then, we have a generator from the **generate** programs that sets up the search space. For example, the rule

```
{ attribute((N,A),E,V): domain((N,A),V) } = 1 :-
  → entity(N,_,E), required((N,A)).
required((mark,type)).
```

requires every mark to have one type from its domain. Finally, Draco loads the hard and soft constraints as **hard** and **soft** programs. Default soft constraint weights are declared in a separate file, and can be loaded and assigned to the constraints when a Draco object is instantiated, which also allows for customized weights. Each program is a dictionary of *blocks* consisting of the constraint and its description. Blocks allow users to pick and choose parts of a program to filter the knowledge base and access documentation. We include unit tests for every constraint and the parser that reads the constraints and documentation from ASP into a Python dictionary.

3.3 Other API and Development Tooling

We provide a well-documented API for interacting with Draco’s knowledge base, encapsulating core utilities for use, extension, and customization. To maintain code quality and to make Draco a viable platform for future research, we follow engineering best practices (i.e., static type analysis, code linting, enforcement of 100% unit test coverage, etc.). Here, we present our Python API and briefly discuss the development tooling we provide. Comprehensive documentation can be found at dig.cmu.edu/draco2.

Specification Renderer: The Draco specification format provides an abstract and machine-readable way to express visualizations. Although, rendered visualization are necessary to efficiently communicate the specifications. We present a default Vega-Lite-based [17] renderer. However, the Draco specification format is generalized and not limited to specific visualization grammars. Therefore, our renderer is extensible such that it comes with an interface from which custom rendering logic can be implemented with the help of our supporting documentation.

Debugging and Constraint Weight Tuning Support: To revise soft constraints and tune weights, the first step is to understand the existing knowledge base by how it is reflected in the recommended results. To support interpreting and inspecting Draco’s output, we provide a *debugger module* (`debug.DracoDebug`) to examine which soft constraints in the knowledge base are violated for a collection of visualizations, and a *plotter module* (`debug.DracoDebugPlotter`) to visualize the violation vectors as shown in Figure 2. One can use these components to determine how to adjust the knowledge base to yield more optimal recommendations, whether by adding or removing constraints, or fine-tuning the weights.

Web Integration: We made Draco easily web-integrable through two main approaches: a) Deploy it as a standalone Python app interfacing via the server module’s web API, or b) Use the Python

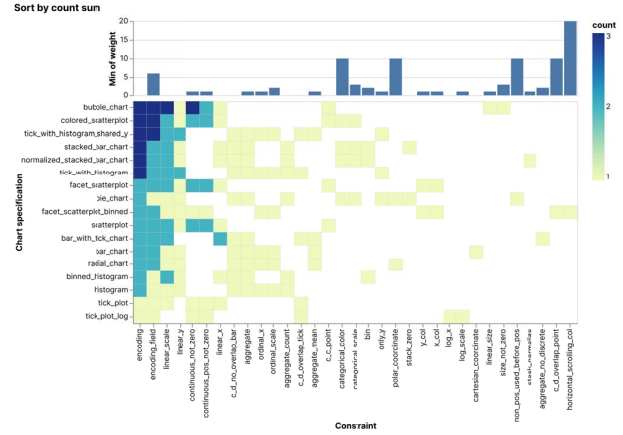


Figure 2: visualizing the violation vectors with `debug.DracoDebugPlotter.create_chart`. It consists of an aligned bar chart displaying constraint weights and a heatmap illustrating constraint violation frequencies for each chart specification.

API directly on the front-end through our WebAssembly distribution with the `draco-pyodide`² package.

3.4 Comparison of Draco 1 and Draco 2

We compare the API implementation, visualization specification format and behavior of Draco 1 and Draco 2 in a Jupyter Notebook³ through a series of examples.

Although similar features are supported by APIs of both versions, Draco 2 proves to be easier to integrate into the Python ecosystem, and it exposes additional features, such as a debugger module. The visualization specification format of the two versions differs vastly. While Draco 1 uses a format tied closely to Vega-Lite, Draco 2 adopts an extensible, visualization-grammar-independent format, built around the idea of using generic entities and attributes.

To compare the behavior of Draco 1 and Draco 2, we selected 100 visualization pairs from the dataset compiled by Kim and Heer [10] for graphical perception study and let the systems rank them using their default knowledge base and constraint weights. Note that even slight weight changes to the knowledge base can lead to very different results, however, the two systems agreed on the ranking 86% of the time, indicating a high behavioral similarity. In each instance of divergent rankings, Draco 2 preferred the visualization which was also deemed better by the participants of Kim and Heer’s study.

4 DEMONSTRATION OF DRACO AS A MODELING TOOL

We demonstrate Draco’s capabilities as an effective tool to generate recommendations, as well as to explore and adapt the knowledge base. We created a general guide for debugging recommendation results⁴. To showcase the procedure and the debugging APIs, we created a Jupyter Notebook⁵ that explores the visualization design space of the Seattle weather dataset⁶. While in principle this exploration would have been possible in the original Draco 1 system, Draco 2’s utilities simplify it significantly.

Draco generates visualization recommendations from incomplete specifications through the `Draco.complete_spec` method. The results might be unsatisfactory for several reasons. When the hard

²npmjs.com/package/draco-pyodide

³dig.cmu.edu/draco2/applications/draco1_vs_draco2.html (permalink)

⁴dig.cmu.edu/draco2/applications/debug_draco.html (permalink)

⁵dig.cmu.edu/draco2/applications/design_space_exploration.html (perma-link)

⁶cdn.jsdelivr.net/npm/vega-datasets@v1.29.0/data/seattle-weather.csv

constraints are not adequate, Draco might output ill-defined results. When the hard constraints are too strong, good candidates might be filtered out without a chance to be ranked with others. Similarly, since the soft constraints and their costs determine the ranking of candidates, there needs to be a suitable degree of differentiation to distinguish between the candidates. With the following example, we demonstrate how to iteratively explore and adjust the Draco knowledge base by modifying the queries and the rules.

Iterating the partial specification query: We start with only the raw dataset input, modeling the common first step in visual data exploration. We load the dataset and generate its schema (name of the columns, their data types, and key statistical properties) as logical facts with the `schema_from_dataframe` function. Then, for the recommendation query, we concatenate the schema with `entity(view,root,v0)` and `entity(mark,v0,m0)`, making sure that the recommendations have at least one view and use at least one mark. We generate and render the top recommendations using `complete_spec` and `AltairRenderer`. The top five results encode the count of records because in general using less encoding is preferred, and they represent an overview given that less information is specified by the query. Two of them encode a non-aggregated second field, `weather`, via columnar faceting. We also observe seemingly identical charts with different costs. These are caused by the fact that there are some entities in the Draco specification such as `"task"` whose value influences the cost computed by Clingo, but does not affect the rendered Vega-Lite specification.

After an initial overview and inspiration, we explore how Draco can be used to create more targeted outputs. To constrain the desired design space, we extend the base input specification with additional facts to target `date` temporal field and `temp_max` numeric field in the target results. We also specify a preference for column-faceted charts, while allowing our tool to determine other details. As a result, we obtain charts faceted by the `weather` field since it is a categorical variable with low cardinality. And the faceted scatter plot without binning is preferred to the faceted tick chart where the `temp_max` field is binned. Draco produces visualizations from partial specification based on the fundamental visualization design guidelines expressed by our knowledge base. We can test more input variations and compare their costs, for example, by forbidding the facet and adding another encoding for the `weather` field that could be encoded in the color channel.

Inspecting the Knowledge Base: To perform a more thorough analysis of the recommendations and to validate them against the design preferences (soft constraints) defined in the knowledge base, we employ Draco's debugger module. The `DracoDebug` module generates a Pandas DataFrame containing the recommendations, the violated soft constraints, and their associated weights. We use `DracoDebugPlotter` to investigate the violation vector and weights interactively. We observe that only a small subset of the defined soft constraints (18 out of 147) impact the recommended charts due to the overlap between them.

Now, we programmatically synthesize a collection of partial specifications to explore more possibilities within the design space. We specify a list of marks `["point", "bar", "line", "rect"]`, fields `["weather", "temp_min", "date"]` and encoding channels `["color", "shape", "size"]` and we enumerate every combination of them in the query to obtain a variety of designs. We observe that Draco uses binning, faceting, stacking, and aggregations such as count and mean in an attempt to recommend meaningful visualizations. We also observe that some mark-field-channel combinations such as `("line", "date", "size")` do not yield any recommendations due to violating a hard-constraint, `size_without_point_text` in this case, defining that encoding data using the size channel only works when using point or text as the mark.

The debugger output indicates that 40 of the defined 147 soft constraints influenced the recommendations from our synthesized

queries, revealing a different design space to target at. From each specification collection, we gather insights of *what combinations can or cannot be rendered, do the rendered charts seem reasonable, what are their costs and violation vectors, and do the costs reflect how they should be ranked in practice*. Through this process, we can confirm how well the knowledge base fits our mental model. Then, either we learn new design guidelines by verifying them with credible sources, or we have detected issues to resolve so that we can improve the knowledge base.

Adjusting the Knowledge Base: Now we demonstrate how to debug constraint logic, tune weights, and discover new rules to add. After analyzing the costs and violation vectors with the heatmap output from the `DracoDebugPlotter`, we may notice that there are unconventional combinations (e.g., `("point", "date", "size")` and `("point", "date", "shape")`) with low costs, meaning that they are likely to be ranked higher among all recommendations with the current knowledge base. By inspecting the pattern in their violation vectors and how they differ from the others, we might detect potential errors in the soft constraint definition or weight of constraints that can be tuned to rank them lower. For example, using the `date` field for `color` or `size` both violate the soft constraint `time_not_x`, which prefers to use the field of type `datetime` on the x-axis. Hence, we could increase its weight and see how that affects the results. We also observe non-expressive designs which share common characteristics not reflected on the violation vector heatmap. This might indicate design space that the existing knowledge base has yet to cover. For example, Draco recommended faceted heatmap designs for the `"rect"` and `"color"` combination, and the design is not discouraged by any existing soft constraints. As we decide to add such a soft constraint, we first assign it a low weight and continue to increase it to test the changes from re-runs.

In conclusion, we demonstrated how Draco can be an easy-to-use modeling tool to interact with the knowledge base and generate visualization recommendations.

5 FUTURE WORK AND CONCLUSION

We believe that Draco is a timely contribution to the visualization research community. And this implementation gets us closer to Draco's original vision of an evolving knowledge base that can be refined, extended, and tested by researchers and practitioners. Draco is already being used as a platform for research. Zeng et al. [26] used Draco to analyze the implication of different graphical perception studies. Recent Large Language Models (LLMs) open new opportunities for visualization recommendation with natural language [3]. Unfortunately, the recommendations from LLMs are hard to explain, steer, and debug. To decouple the interpretation of the natural language input and recommendation, we could use an LLM and Draco together. The LLM could generate a partial specification and then Draco could complete the specification and generate a visualization. Another application of Draco could be as a way to embed visualizations in machine learning models. The violation of soft constraints of a particular visualization forms a vector that could be used as a feature in a machine learning model. These are just some of the potential projects we envision for Draco: others may extend its model to fine-grained task taxonomies or interactive charts.

In conclusion, we present a major improvement over Draco to make it more self-contained, well-documented and extensible. We believe that these improvements make Draco a solid foundation for future research.

ACKNOWLEDGMENTS

We thank our labs for their feedback on this system and paper, especially Manfred Klaffenböck, Torsten Möller, Halden Lin and Aameya Patil.

REFERENCES

- [1] Q. Chen, F. Sun, X. Xu, Z. Chen, J. Wang, and N. Cao. Vizlinter: A linter and fixer framework for data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):206–216, 2022. doi: 10.1109/TVCG.2021.3114804
- [2] Ç. Demiralp, P. J. Haas, S. Parthasarathy, and T. Pedapati. Foresight: Recommending visual insights. *Proceedings of the VLDB Endowment*, 10(12):1937–1940, Aug. 2017. doi: 10.14778/3137765.3137813
- [3] V. Dibia. Lida: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. *arXiv preprint arXiv:2303.02927*, 2023.
- [4] M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub. Clingo = ASP + control: Preliminary report. *CoRR*, abs/1405.3694, 2014.
- [5] M. Gebser, R. Kaminski, and T. Schaub. Complex optimization in answer set programming. *Theory and Practice of Logic Programming*, 11(4-5):821–839, 2011. doi: 10.1017/S1471068411000329
- [6] M. Gebser, B. Kaufmann, R. Kaminski, M. Ostrowski, T. Schaub, and M. Schneider. Potassco: The potsdam answer set solving collection. *AI Communications*, 24(2):107–124, apr 2011. doi: 10.3233/AIC-2011-0491
- [7] A. K. Hopkins, M. Correll, and A. Satyanarayan. Visualint: Sketchy in situ annotations of chart construction errors. In *Computer Graphics Forum*, vol. 39, pp. 219–228. Wiley Online Library, 2020.
- [8] K. Hu, M. Bakker, S. Li, T. Kraska, and C. Hidalgo. VizML: A machine learning approach to visualization recommendation. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '19, pp. 1–12. Association for Computing Machinery, New York, NY, USA, 2019. doi: 10.1145/3290605.3300358
- [9] A. Key, B. Howe, D. Perry, and C. Aragon. VizDeck: Self-organizing dashboards for visual analytics. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, SIGMOD '12, p. 681–684. Association for Computing Machinery, New York, NY, USA, 2012. doi: 10.1145/2213836.2213931
- [10] Y. Kim and J. Heer. Assessing effects of task and data distribution on the effectiveness of visual encodings. *Computer Graphics Forum*, 37(3):157–167, 2018. doi: 10.1111/cgf.13409
- [11] H. Li, Y. Wang, S. Zhang, Y. Song, and H. Qu. KG4Vis: A knowledge graph-based approach for visualization recommendation. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):195–205, 2022. doi: 10.1109/TVCG.2021.3114863
- [12] Y. Luo, X. Qin, N. Tang, and G. Li. DeepEye: Towards automatic data visualization. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, pp. 101–112, April 2018. doi: 10.1109/ICDE.2018.00019
- [13] J. Mackinlay. Automating the design of graphical presentations of relational information. *ACM Transactions on Graphics*, 5(2):110–141, Apr. 1986. doi: 10.1145/22949.22950
- [14] J. Mackinlay, P. Hanrahan, and C. Stolte. Show Me: Automatic presentation for visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1137–1144, Nov 2007. doi: 10.1109/TVCG.2007.70594
- [15] A. M. McNutt and G. L. Kindlmann. Linting for visualization: Towards a practical automated visualization guidance system. 2018.
- [16] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in draco. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):438–448, Jan 2019. doi: 10.1109/TVCG.2018.2865240
- [17] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, Jan 2017. doi: 10.1109/TVCG.2016.2599030
- [18] T. Siddiqui, J. Lee, A. Kim, E. Xue, X. Yu, S. Zou, L. Guo, C. Liu, C. Wang, K. Karahalios, and A. G. Parameswaran. Fast-forwarding to desired visualizations with zenvisage. In *8th Biennial Conference on Innovative Data Systems Research, CIDR 2017, Chaminade, CA, USA, January 8-11, 2017, Online Proceedings*. www.cidrdb.org, 2017.
- [19] M. Vartak, S. Rahman, S. Madden, A. Parameswaran, and N. Polyzotis. SeeDB: Efficient data-driven visualization recommendations to support visual analytics. *Proceedings of the VLDB Endowment*, 8(13):2182–2193, Sept. 2015. doi: 10.14778/2831360.2831371
- [20] L. Wilkinson. *The Grammar of Graphics (Statistics and Computing)*. Springer-Verlag, Berlin, Heidelberg, 2005.
- [21] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Towards a general-purpose query language for visualization recommendation. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics, HILDA '16*, pp. 4:1–4:6. ACM, New York, NY, USA, 2016. doi: 10.1145/2939502.2939506
- [22] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. vol. 22, pp. 649–658, Jan 2016. doi: 10.1109/TVCG.2015.2467191
- [23] K. Wongsuphasawat, Z. Qu, D. Moritz, R. Chang, F. Ouk, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager 2: Augmenting visual analysis with partial view specifications. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '17, p. 2648–2659. Association for Computing Machinery, New York, NY, USA, 2017. doi: 10.1145/3025453.3025768
- [24] Z. Zeng and L. Battle. A review and collation of graphical perception knowledge for visualization recommendation. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '23. ACM, New York, NY, USA, 2023. doi: 10.1145/3544548.3581349
- [25] Z. Zeng, P. Moh, F. Du, J. Hoffswell, T. Y. Lee, S. Malik, E. Koh, and L. Battle. An evaluation-focused framework for visualization recommendation algorithms. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):346–356, 2022. doi: 10.1109/TVCG.2021.3114814
- [26] Z. Zeng, J. Yang, D. Moritz, J. Heer, and L. Battle. Too many cooks: Exploring how graphical perception studies influence visualization recommendations in draco. *IEEE Transactions on Visualization and Computer Graphics*, 2023.
- [27] S. Zhu, G. Sun, Q. Jiang, M. Zha, and R. Liang. A survey on automatic infographics and visualization recommendations. *Visual Informatics*, 4(3):24–40, 2020. doi: 10.1016/j.visinf.2020.07.002